

Lost in Transition

The base of the game is describing words and guessing words. To each player it will be given a word and they will try to describe it in 10 words or less without mentioning the actual word. That description then will be given to another player and that player will try to guess what word is it and based on that guess another player will try to describe that word. The game will keep going for x rounds as x players are and at the end it will be shown how much the original word changed. Every round the player will describe the word using less words going from 10 to 7 (if there is 4 players) to 5 (if there is 6 players) to 3 (if there is 8 players).

The default words will be stored in an array, the array will be as big as the number of players there is. After prompting to the user to describe the word given, they will be stored in another array that with a sort algorithm and replacement will be moved around to the left so every player will have the answer of the player on their right and the game is going to keep moving on until the original words come back to where they were initially.

The winner will be selected by which of the final words is the closest to the original. The way that the winner will be selected is by using some tiers of algorithms. Tier 1 is Exact match, if the word is the exact one then they directly win. Tier 2 is Levenshtein Distance, this one measures how many single characters modification separate two strings (the original and the final word). Tier 3 Semantic Similarity, this would compare the meaning of each word and determine how much they have drifted from the original and assign a score. The final words will be compared to the original by using the website: <https://www.onelook.com/thesaurus/?loc=lhp>, where the closest to 1 the word is the most similar it is.

Network Communication: It uses Network Communication because it needs multiple players connected from their device to the server where the game is hosted.

Concurrency: The server will handle multiple entries at the same time, since there are multiple players and every player will be answering their own question at the same time and will be adding their own inputs all at the same time. Each client connection will be handled by its own thread, so no player has to wait for another to finish before their input is processed.

Shared State Management: The server will maintain a shared game state containing each player's word, descriptions, and guesses. Since players submit simultaneously, access to this data will be synchronized blocks to prevent race conditions and data mix-ups.

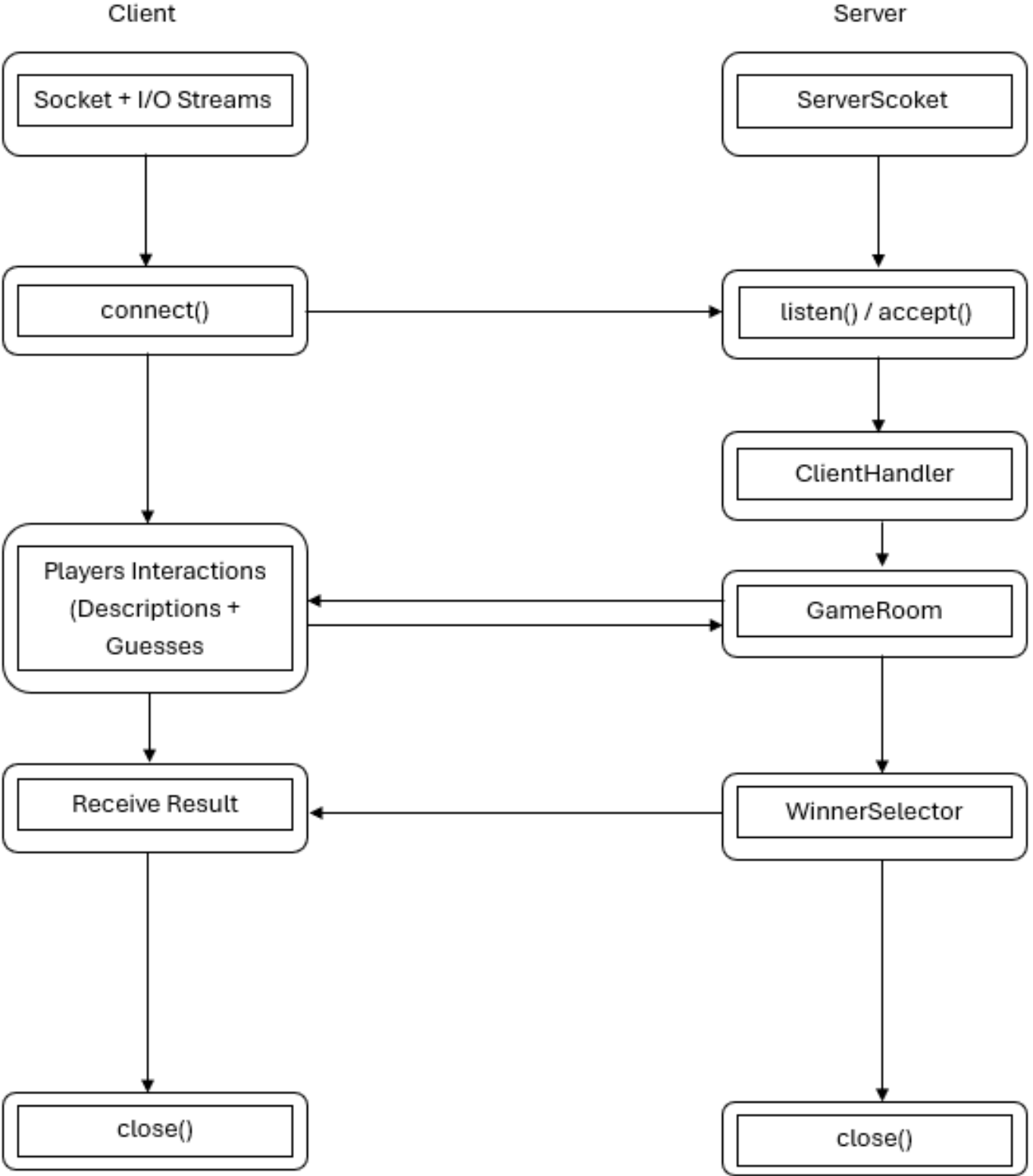
Resilience and Error Handling: If a player disconnects unexpectedly, the server will catch the IOException, remove that player from the session, and allow the remaining players to

continue. All network operations will be wrapped in try-catch blocks so the server never crashes due to a single client's failure.

Measurable Deliveries

- 1) A fully functional multiplayer Word Drift game server that supports at least 3 simultaneous players connected over a network, with a complete round cycle (word assignment to description to passing to guessing) executing without errors.
- 2) A real-time shared game state manager ensures all connected clients see consistent round data (current word limits, player turns, submitted descriptions) with handling for at least one failure case (a player disconnecting mid-round).
- 3) A winner-selection system implemented and tested in Java, using exact match, Levenshtein distance, and Datamuse-powered meaning similarity to score final guesses against original words.

Flow Chart



Basic Code

GameServer.java: Manage the ServerSocket and client connection

GameRoom.java: Holds game state, players and round logic. Will be used as the main class.

Player.java: Represents a connected player

WordChain.java: Tracks the word/description passing chain

WinnerSelector.java: Determines who wins and shows it to the players.

```
import java.net.*;

public class GameServer {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
        int port = 1728;
        ServerSocket listener;
        Socket connection;

        try {
            listener = new ServerSocket(port);
            System.out.println("Word Drift server started on port " + listener.
                getLocalPort());

            connection = listener.accept();
            listener.close();
            System.out.println("Client connected: " + connection.getInetAddress
                ());
        } catch (Exception e) {
            System.out.println("ERROR: Connection could not be established.");
            return;
        }
    }
}
```